

PORTADA

Calidad de Software

Semana 9 — El proceso de pruebas, el ciclo de vida del defecto y su reporte

Universidad de Antioquía · Ingeniería de Sistemas



ENCUADRE

Hoy se entrega el Trabajo 1

Al inicio de esta semana se entrega el **Trabajo 1 (25 %)**: plan de pruebas y diseño de casos, aplicando todo lo visto en las Unidades 1, 2 y 4 al proyecto de Fábrica-Escuela.

Con esa entrega arranca la **Unidad 3 — Implementación del proceso de aseguramiento**: de aquí en adelante, el curso deja de hablar en abstracto y empieza a ejecutar el proceso completo, de principio a fin.



REPASO RELÁMPAGO

Lo que ya tenemos de la semana 8

- **Caja negra** parte de la especificación; **caja blanca**, del código.
- Particiones de equivalencia, valores límite, tablas de decisión y transición de estados diseñan casos de forma sistemática.
- La **pirámide de Cohn**: muchas pruebas rápidas en la base, pocas lentas en la punta.



TRANSICIÓN

Hoy: el proceso completo, de principio a fin

Ya tenemos todas las piezas sueltas: se planea (semana 6), se decide qué tipos aplican (semana 7), se diseñan los casos (semana 8). Hoy se unen en un solo proceso — y se ve qué pasa **después**, cuando una prueba efectivamente encuentra algo.



CONCEPTO

El proceso de pruebas, de un vistazo

1. **Planear** — alcance, criterios, riesgos, recursos (semana 6).
2. **Diseñar** — casos con las técnicas de caja negra y blanca (semana 8).
3. **Ejecutar** — correr los casos contra el sistema real.
4. **Reportar** — documentar cada discrepancia encontrada como un defecto.
5. **Verificar y cerrar** — confirmar la corrección y cerrar el ciclo.

Los pasos 4 y 5 son el tema de hoy — el resto ya se vio en semanas anteriores.

TRANSICIÓN

Qué pasa cuando una prueba falla

Encontrar una discrepancia es solo el principio. Ese hallazgo tiene que viajar de quien lo encontró a quien lo corrige, y de ahí de vuelta — con un registro claro de en qué punto de ese viaje está en cada momento.



CONCEPTO · RECORDATORIO

Defecto, otra vez

Semana 1: un **defecto** es la huella que un error humano deja en un artefacto — código, configuración, documento. Es distinto de la **falla**, que es el comportamiento incorrecto observable cuando ese defecto se ejecuta.

Hoy el defecto deja de ser solo un concepto y se vuelve un objeto que se gestiona: tiene un estado, cambia de dueño, y sigue un camino predecible desde que se encuentra hasta que se cierra.

CONCEPTO

El ciclo de vida del defecto

Es el conjunto de **estados por los que pasa un defecto**, desde que se reporta hasta que se cierra, y las reglas que dicen qué transiciones son válidas:

- **Nuevo:** recién reportado, sin revisar todavía.
- **Asignado:** un desarrollador lo tiene a cargo.
- **En progreso:** se está corrigiendo.
- **Corregido / Listo para retest:** el desarrollador dice que ya está solucionado.
- **Cerrado:** quien lo reportó confirmó que la corrección funciona.
- **Reabierto:** la corrección no funcionó — vuelve a "En progreso".
- **Rechazado / Diferido:** no se considera un defecto, o se pospone a propósito.

CONCEPTO

Quién puede mover cada estado

Transición	Quién la hace
Nuevo → Asignado	Líder técnico o el equipo, al triar el defecto
Asignado → En progreso	El desarrollador, al empezar a trabajarlo
En progreso → Corregido	El desarrollador, al terminar la corrección
Corregido → Cerrado	Quien reportó, tras confirmar que ya no falla
Corregido → Reabierto	Quien reportó, si la falla sigue presente

Nadie cierra su propio defecto — cerrarlo le corresponde a quien lo encontró.

EJEMPLO

Un defecto recorriendo el ciclo completo

1. QA reporta: "El botón Transferir permite montos negativos" → **Nuevo**.
2. El líder técnico lo revisa y lo asigna a un desarrollador → **Asignado**.
3. El desarrollador empieza a trabajarlo → **En progreso**.
4. Agrega la validación y sube el cambio → **Corregido**.
5. QA vuelve a probar con un monto negativo: el sistema sigue permitiéndolo en un segundo formulario que no se corrigió → **Reabierto**.
6. El desarrollador corrige también ese formulario → **Corregido**.
7. QA confirma que ya no falla en ningún formulario → **Cerrado**.

TRANSICIÓN

No todos los defectos son iguales

Un defecto en "Nuevo" necesita que alguien decida dos cosas antes de asignarlo: qué tan grave es, y qué tan urgente es corregirlo. No son la misma pregunta.



CONCEPTO · RECORDATORIO

Severidad, otra vez

Semana 4: la **severidad** mide **qué tanto daño causa** un defecto si no se corrige — Crítica, Alta, Media, Baja. Es una propiedad técnica del defecto: la decide quien lo encuentra, mirando el impacto real en el sistema.



CONCEPTO

Qué es la prioridad

La **prioridad** mide **qué tan pronto** debe corregirse un defecto, en relación con todo lo demás que compite por el tiempo del equipo. A diferencia de la severidad, **es una decisión de negocio**, no solo técnica — la toma quien planea el trabajo, considerando plazos, visibilidad del defecto para el cliente, y esfuerzo disponible.

Un defecto puede ser grave y no urgente, o leve y muy urgente — por eso hacen falta las dos escalas, no una sola.



CONCEPTO

Severidad y prioridad, combinadas

	Prioridad alta	Prioridad baja
Severidad alta	El sistema de pagos rechaza todas las transacciones	Un módulo interno, poco usado, calcula mal un reporte histórico
Severidad baja	El logo de la empresa se ve mal justo antes del lanzamiento de marca	Un texto de ayuda tiene una errata menor

La celda "severidad baja, prioridad alta" es la que más sorprende — y la más fácil de justificar mal si no se separan las dos escalas.

TRANSICIÓN

Cómo se documenta todo esto

Un defecto que solo existe en la cabeza de quien lo encontró no sirve de nada. Necesita quedar escrito de una forma que cualquier otra persona del equipo pueda entender y reproducir, sin tener que preguntar.



CONCEPTO · HERRAMIENTA

Qué es un reporte de defectos

Es el documento —normalmente un ticket en una herramienta como **Jira**— que registra un defecto de forma que se pueda **rastrear, priorizar y verificar** a lo largo de todo su ciclo de vida. Es la unidad básica de comunicación entre quien encuentra el problema y quien lo corrige.



CONCEPTO

Qué contiene un buen reporte

- **Título claro:** resume el problema en una línea — no "no funciona", sino qué exactamente no funciona.
- **Pasos de reproducción:** numerados, específicos, sin dar nada por sentado.
- **Resultado esperado vs. observado:** qué debía pasar y qué pasó en realidad.
- **Severidad y prioridad.**
- **Ambiente:** navegador, sistema operativo, versión, credencial usada.
- **Evidencia:** captura de pantalla, video, o log — con fecha y hora visibles.

CONTRAEJEMPLO

Un reporte que no sirve

Mal: *"El carrito de compras no funciona bien, revisar cuando puedan."*

- No dice qué acción se hizo ni en qué pantalla.
- No dice qué se esperaba ni qué pasó realmente.
- Sin severidad, sin prioridad, sin evidencia — nadie puede reproducirlo ni priorizarlo.

Mejor: *"Al agregar 2 unidades del mismo producto al carrito, el subtotal muestra el precio de 1 unidad en vez de 2. Esperado: subtotal = precio × cantidad. Severidad: Alta. Prioridad: Alta (afecta el cobro real)."*

INTERACCIÓN

Ejercicio rápido: severidad y prioridad

5 minutos — clasifiquen cada caso:

1. El sitio se cae por completo para todos los usuarios.
2. Un descuento promocional del 50 % no se aplica, y la promoción termina mañana.
3. Un ícono decorativo en el pie de página está pixelado.

Este ejercicio es el calentamiento del taller de hoy.

CONCEPTO EN DISPUTA

¿Quién decide la prioridad?

Es tentador que el mismo tester que asigna la severidad también asigne la prioridad —al fin y al cabo, ya conoce el defecto a fondo. El problema: la prioridad depende de información de negocio que un tester no siempre tiene —plazos con el cliente, visibilidad de marca, presión comercial.

La práctica más sólida separa los roles: **el tester propone** una prioridad inicial basada en severidad, pero **el equipo o el responsable del producto la confirma** en la reunión de triage, con el contexto completo.



SÍNTESIS

Lo que hay que llevarse de hoy

1. El proceso de pruebas no termina al ejecutar un caso — **reportar y cerrar** son parte de él.
2. Un defecto tiene un **ciclo de vida**: Nuevo → Asignado → En progreso → Corregido → Cerrado (o Reabierto).
3. **Severidad** mide daño; **prioridad** mide urgencia — son ejes independientes.
4. Un buen reporte de defectos es **reproducible**: pasos claros, esperado vs. observado, evidencia con fecha y hora.
5. Cerrar un defecto le corresponde a **quien lo encontró**, no a quien lo corrigió.



INSTRUCCIONES

Taller de hoy: reportar 6 defectos en ParaBank

2 horas, en sala de Zoom asignada — equipo fijo de Fábrica-Escuela

Con la credencial asignada, cada equipo debe encontrar y reportar **6 defectos** sobre [ParaBank](#), cada uno con:

- **Pasos de reproducción** claros y numerados.
- **Severidad y prioridad**, cada una justificada por separado.
- **Evidencia** (captura o video) con **marca de fecha y hora de la sesión** visible.

La entrega cierra al terminar las dos horas. Recuerden: al enviar, cada integrante reporta en privado quién trabajó.



CIERRE

Para la próxima semana

Antes de la clase de la semana 10, hay que llegar con esto leído:

- Pruebas unitarias. Frameworks (JUnit / pytest). Test doubles: mocks, stubs, fakes.

Seguimos con los equipos fijos de **Fábrica-Escuela**, dentro de la Unidad 3. El taller de la semana 10: suite de pruebas unitarias sobre una regla de negocio, con casos límite de la semana 8 y al menos un test double.

